

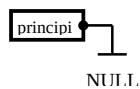
LISTA SIMPLEMENTE ENLAZADA

Una lista es un tipo de estructura de datos que forma una cadena mediante un enlace entre un dato y el siguiente. Como existe un enlace entre los diferentes datos no esta acotada la cantidad de posibles datos a ser procesados.

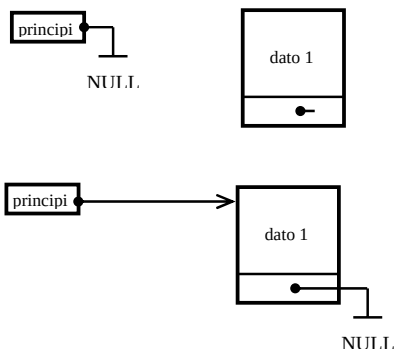
El límite máximo lo impone la memoria, debido a que cada conjunto de datos debe ubicarse en una posición de memoria distinta, esta debe ser solicitada mediante el uso de las funciones de asignación dinámica (por ej. malloc()).

Para poder utilizar este tipo de estructura se deben agrupar los datos con un puntero, esto se realiza mediante el uso de una estructura y dentro de ella un puntero a la misma. Además debemos agregar un indicativo al principio de los datos, esto lo debemos realizar mediante un puntero a estructura.

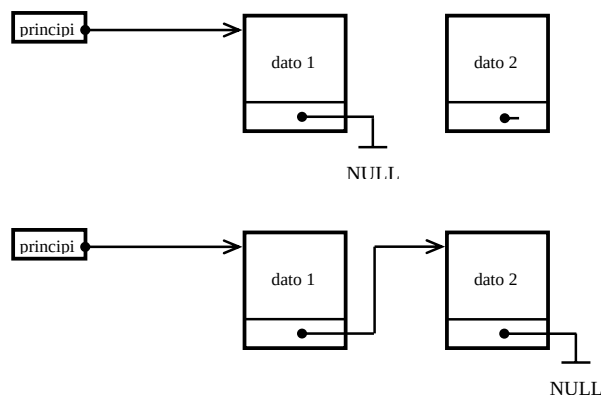
Antes de comenzar a crear la lista, el puntero que indica el comienzo, debe apuntar a un valor NULL, que indica que la lista esta vacía. Este caso lo vemos en la figura.



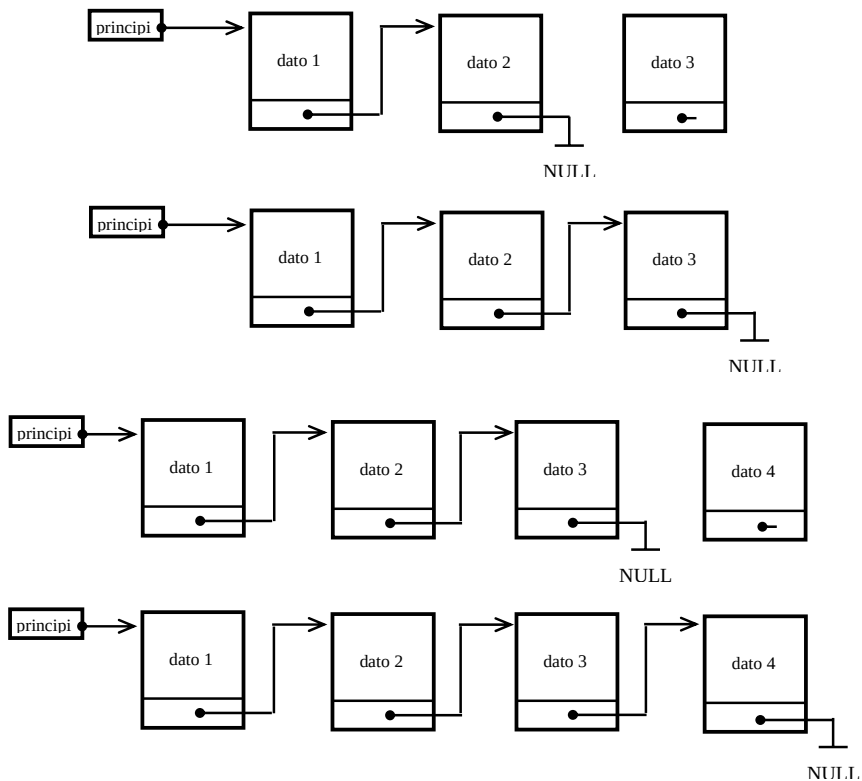
Cuando se genera el primer conjunto de datos el puntero de comienzo debe apuntar a este conjunto y el puntero del dato deberá apuntar a NULL, indicando el fin de la lista. Como podemos visualizar en el siguiente dibujo.



El siguiente elemento puede ubicarse a continuación del último para lo cual el puntero del conjunto de datos que existe en la lista se deberá apuntar a la nueva estructura y el puntero de esta apuntará a NULL.



Así sucesivamente pueden incluirse los datos uno por uno, veamos como se irían incluyendo un tercer dato a la lista y luego un cuarto.



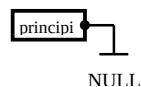
De esta forma podemos ir haciendo crecer la lista a medida que se adicionan los datos, pero de esta manera los datos están desordenados.

La manera vista anteriormente no es la más eficiente para manejar los datos, pero sí es sencilla; una mejor ubicación de los datos sería, ordenarlos según una determinada llave. Este método permite insertar cada conjunto de datos en el lugar adecuado según el orden deseado.

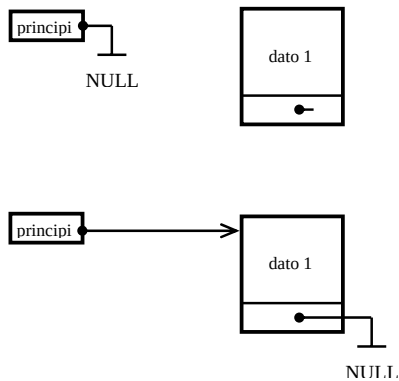
Aquí se pueden presentar tres posibilidades:

- Insertar como primer elemento.
- Insertar como último elemento.
- Insertar como un elemento intermedio.

Siempre que la lista este vacía el puntero de comienzo estará apuntando a NULL, como en el caso anterior.



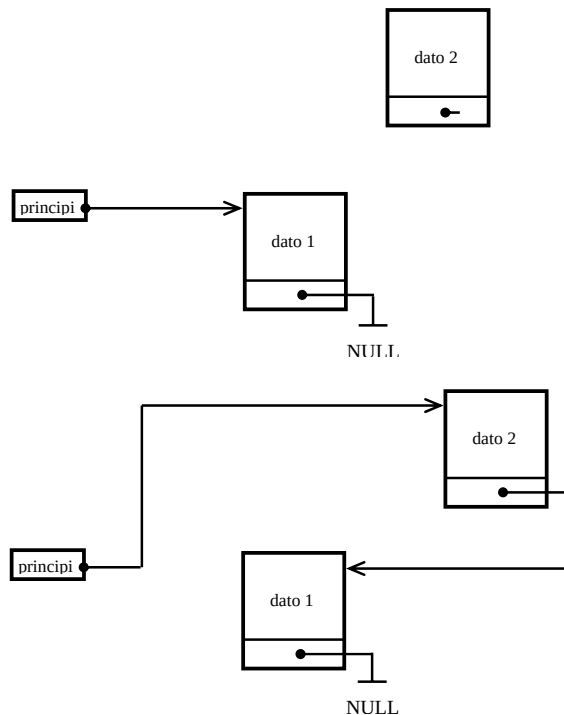
El primer conjunto de datos estaría apuntado por el puntero de inicio, y lo podemos visualizar en el siguiente esquema.



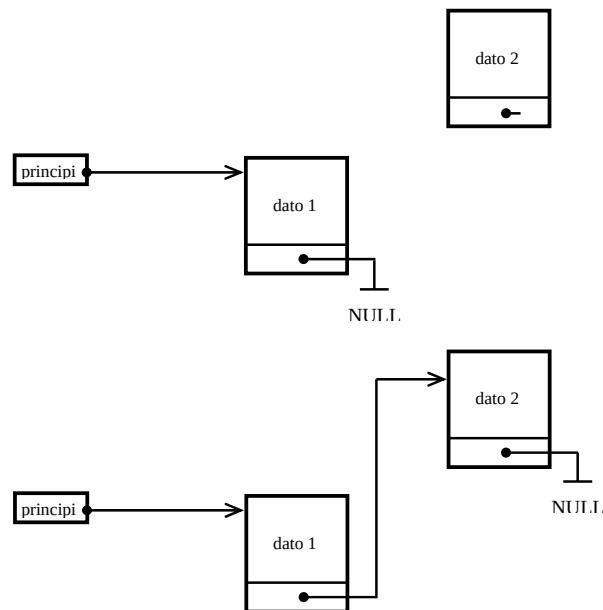
El segundo podría tener dos posibles posiciones para ser insertado:

- Como nuevo primer elemento.
- Como último elemento.

En el primer caso el puntero de inicio debe apuntar al nuevo elemento, el puntero del nuevo elemento debe apuntar al elemento que estaba en la lista y el puntero de este deberá apuntar a NULL.



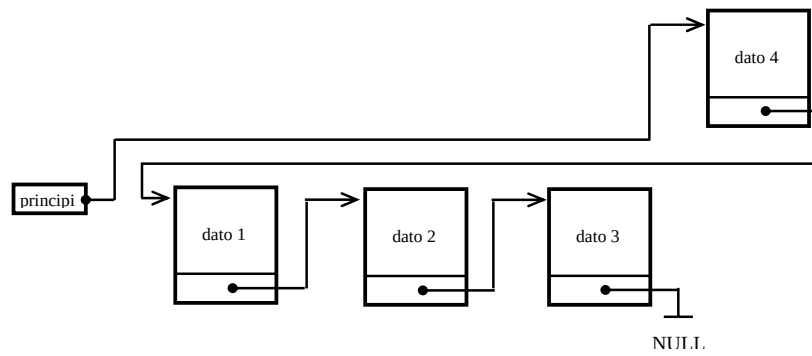
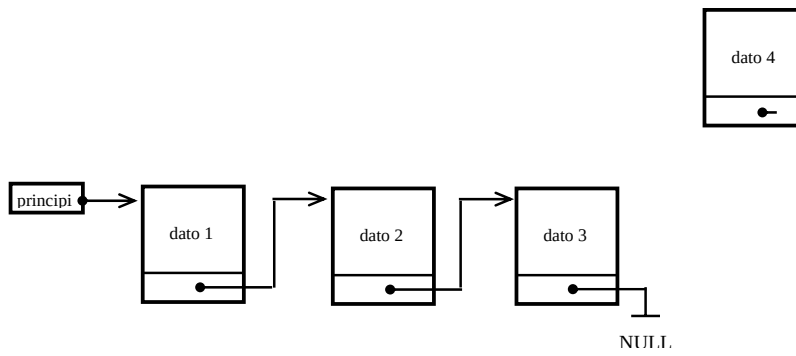
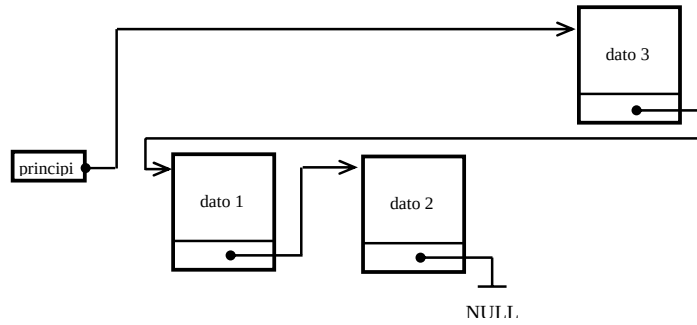
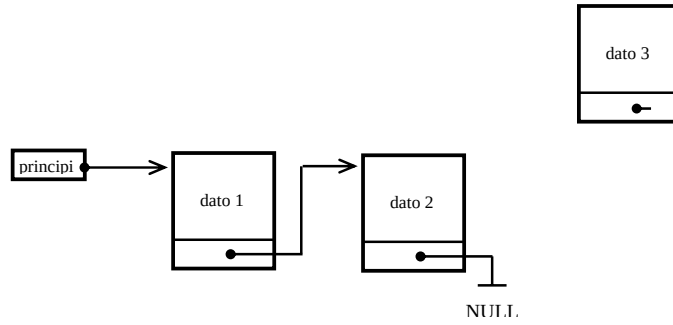
La otra posibilidad es la de que el nuevo elemento se sitúe al final de la lista entonces el dato que ya pertenecía a ella quedará en la posición actual y apuntará al nuevo conjunto, a su vez el puntero de este nuevo conjunto apuntará a NULL. Este caso es igual al visto anteriormente.



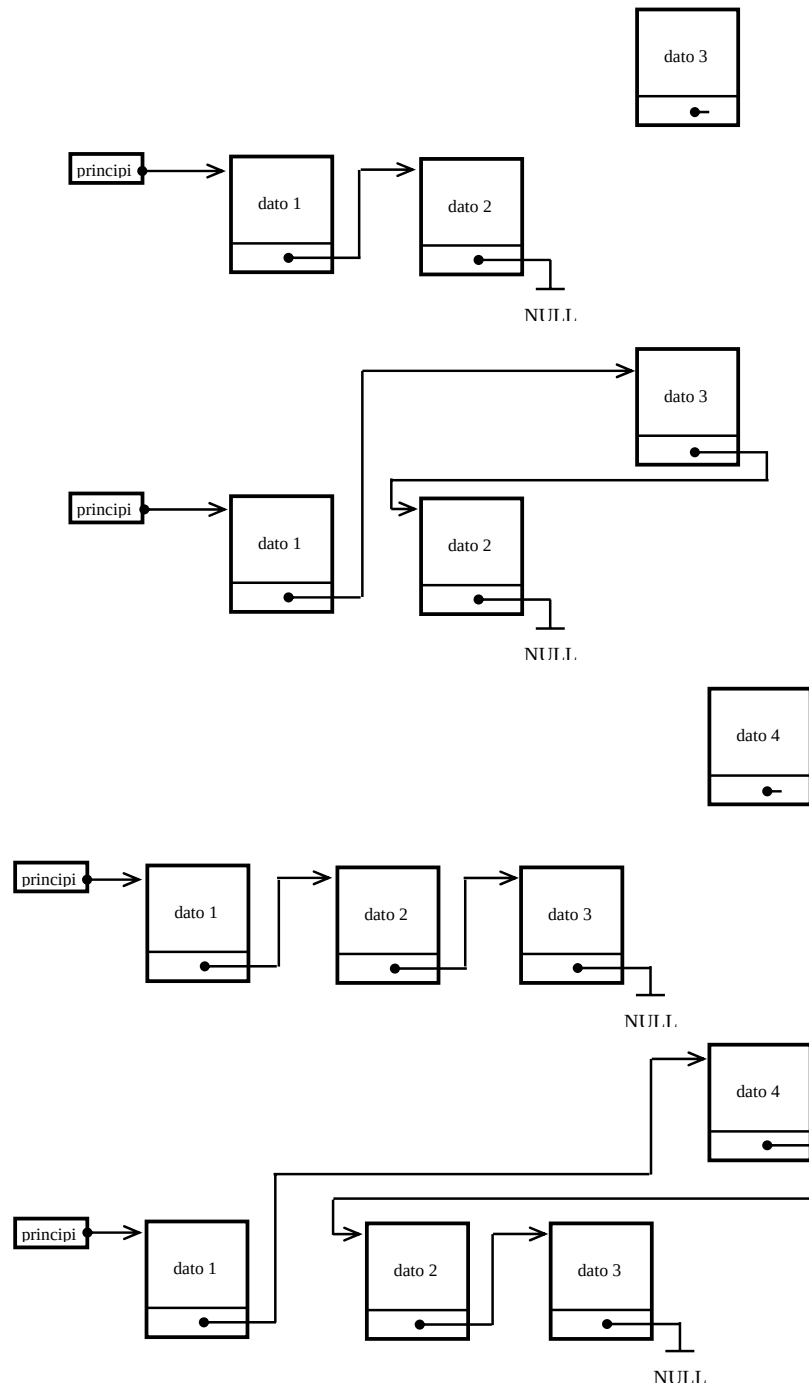
Desde el tercer elemento a insertar, pueden producirse las tres posibilidades que habíamos mencionado inicialmente:

- Como nuevo primer elemento.
- Como elemento intermedio.
- Como último elemento.

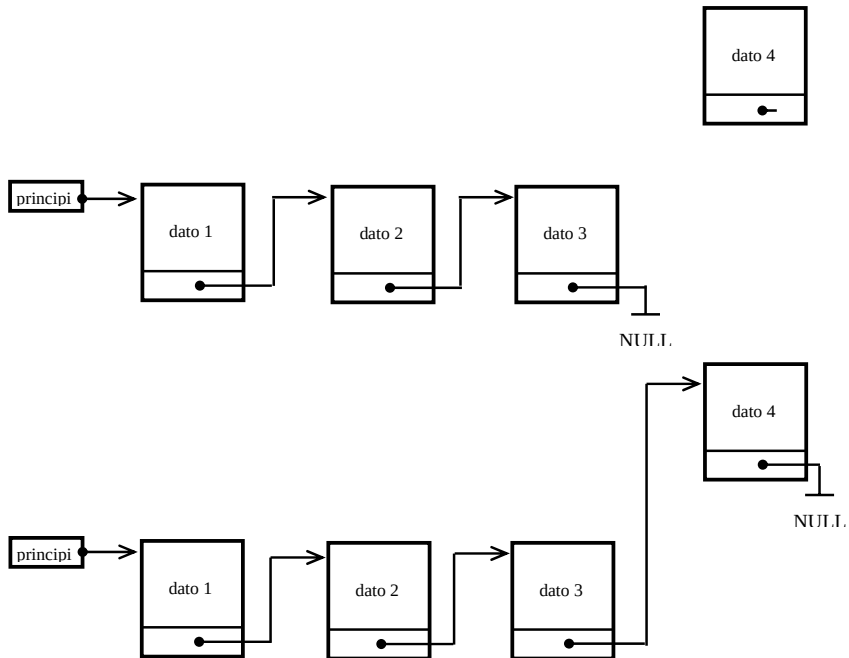
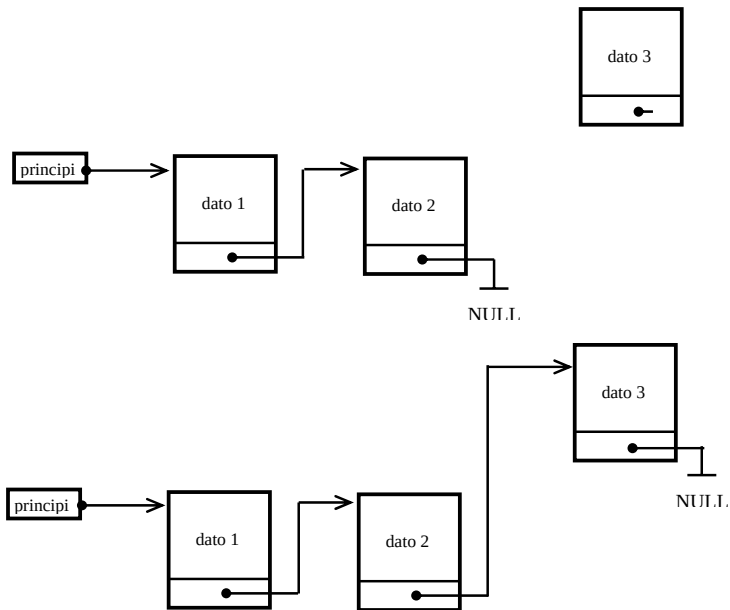
Cuando sucede que el nuevo elemento debe ser el primer elemento el indicador de principio deberá apuntar a este nuevo elemento y el indicador del elemento siguiente en el nuevo deberá apuntar al elemento que era el primero.



Otra posible ubicación es entre dos elementos existentes, entonces se debe cambiar el enlace del elemento anterior hacia el nuevo y el enlace del nuevo apuntará al posterior.



Por último puede ocurrir que el elemento a ser insertado debe ubicarse al final de la lista, por lo tanto el puntero siguiente del elemento final debe ahora apuntar al nuevo elemento y el puntero siguiente del nuevo debe apuntar a NULL.



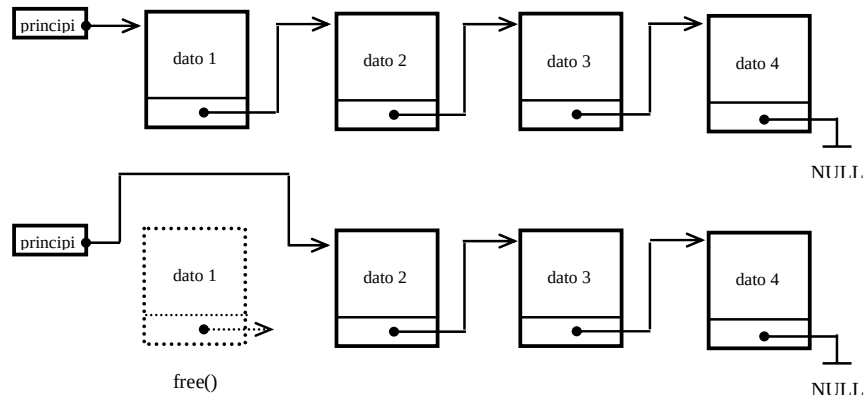
Para leer los elementos de la lista solo se debe comenzar desde la dirección que apunta el indicador de comienzo y luego de presentar los datos, solo se debe actualizar el puntero con el valor del puntero siguiente de la estructura hasta llegar a encontrar el valor NULL.

Cuando se debe buscar un determinado valor en la lista se debe hacer en forma secuencial por lo que hay que comenzar desde el valor que indica el puntero de comienzo hasta encontrar el valor deseado o hasta encontrar el NULL, que indicaría que no se ha encontrado el valor buscado.

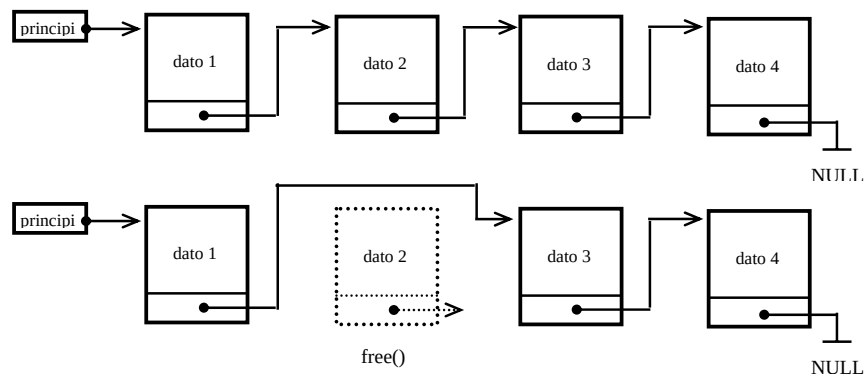
Cuando se desee borrar un elemento encontrado en la lista pueden ocurrir tres casos:

- Borrar el primer elemento.
- Borrar un elemento intermedio.
- Borrar el último elemento.

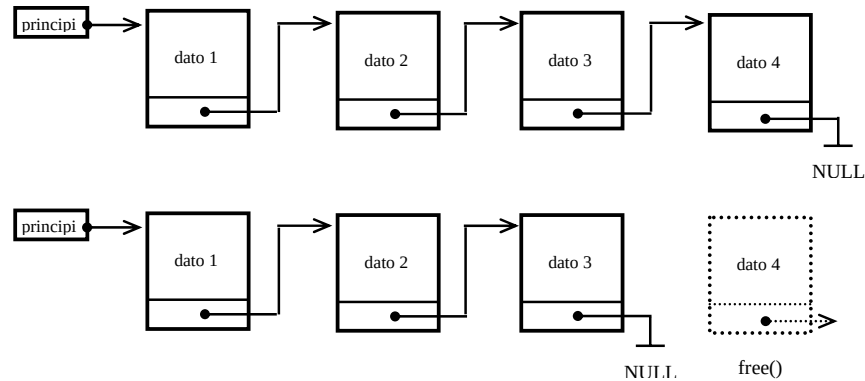
Si es el primer elemento se debe cambiar el puntero de comienzo a la dirección del segundo elemento. Luego se debe proceder a liberar la memoria del elemento borrado mediante el uso de la función `free()`.



En caso de que sea un elemento intermedio se debe hacer que el puntero siguiente del elemento anterior apunte al elemento siguiente al que será borrado y luego liberar la memoria de este.



Si el elemento a eliminar esta ubicado al final de la lista debemos proceder de la siguiente forma: el puntero siguiente del elemento anterior debe apuntar a NULL y luego liberar la memoria del elemento a borrar.



Ejemplo

```

1. //-----
2. #include<stdio.h>
3. #include<stdlib.h>
4. #include<conio.h>
5. //-----
6. struct Lis
7. {
8.     int Dato;
9.     struct Lis *Sig;
10. };
11. //-----
12. void Ingresa(struct Lis **);
13. void Lee(struct Lis **);
14. void Borra(struct Lis **);
15. void Elimina(struct Lis **);
16. //-----
17. int main(void)
18. {
19.     int Opcion;
20.     struct Lis *Inicio=NULL;
21.     for(;;)
22.     {
23.         system("cls");
24.         printf("\n1 - Ingresa datos\n");
25.         printf("2 - Lee datos\n");
26.         printf("3 - Borra datos\n");
27.         printf("4 - Salir\n\n");
28.         printf("Ingresa una opcion ( 1 - 4 ) : ");
29.         Opcion=getche();
30.         switch(Opcion)
31.         {
32.             case '1':
33.                 Ingresa(&Inicio);
34.                 break;
35.             case '2':
36.                 Lee(&Inicio);
37.                 break;
38.             case '3':
39.                 Borra(&Inicio);
40.                 break;

```

```

41.         case '4':
42.             Elimina(&Inicio);
43.             exit(0);
44.         }
45.     }
46.     return 0;
47. }
48. //-----
49. void Ingresa(struct Lis **Principio)
50. {
51.     struct Lis *Actual=*Principio,*Nuevo;
52.     system("cls");
53.     if(!(Nuevo=(struct Lis *)malloc(sizeof(struct Lis))))
54.     {
55.         system("cls");
56.         printf("\n\nNo hay memoria disponible\n");
57.         printf("\n\nPresione una tecla para continuar\n");
58.         getch();
59.         return;
60.     }
61.     printf("\n\nIngrese el dato : ");
62.     scanf("%d",&Nuevo->Dato);
63.     Nuevo->Sig=NULL;
64.     if(!*Principio)
65.     {
66.         *Principio=Nuevo;
67.         return;
68.     }
69.     while(Actual->Sig)
70.         Actual=Actual->Sig;
71.     Actual->Sig=Nuevo;
72. }
73. //-----
74. void Lee(struct Lis **Principio)
75. {
76.     struct Lis *Actual=*Principio;
77.     system("cls");
78.     while(Actual)
79.     {
80.         printf("\nEl dato es : %d",Actual->Dato);
81.         Actual=Actual->Sig;
82.     }
83.     printf("\n\nPresione una tecla para continuar\n");
84.     getch();
85. }
86. //-----
87. void Borra(struct Lis **Principio)
88. {
89.     struct Lis *Actual=*Principio,*Anterior=*Principio;
90.     int Valor;
91.     system("cls");
92.     printf("\nIngrese el dato a borrar : ");
93.     scanf("%d",&Valor);
94.     while(Actual&&(Valor!=Actual->Dato))
95.     {
96.         Anterior=Actual;
97.         Actual=Actual->Sig;
98.     }
99.     if(Actual&&(*Principio==Actual))

```

```

100.  {
101.      *Principio=Actual->Sig;
102.      free(Actual);
103.      return;
104.  }
105.  if(Actual)
106.  {
107.      Anterior->Sig=Actual->Sig;
108.      free(Actual);
109.      return;
110.  }
111.  system("cls");
112.  printf("\nEl dato no ha sido encontrado");
113.  printf("\n\nPresione una tecla para continuar\n");
114.  getch();
115. }
116. //-----
117. void Elimina(struct Lis **Principio)
118. {
119.     struct Lis *Actual;
120.     while(*Principio)
121.     {
122.         Actual=*Principio;
123.         *Principio=( *Principio)->Sig;
124.         free(Actual);
125.     }
126. }
127. //-----

```

En este ejemplo tenemos una aplicación de una lista simplemente enlazada, la que permite ingresar un dato.

El dato está contenido en una estructura llamada Lis que se define entre las líneas 6 a 10, contiene un puntero a struct Lis llamado Sig que va a ser el encargado de apuntar al elemento siguiente de la lista. Cabe destacar que el tamaño que ocupa un puntero como el mencionado es siempre de 4 bytes, por lo tanto cuando se define la estructura lo que se hace es solamente guardar este espacio para el puntero, el tipo al que apunta se lo coloca para determinar su forma y además para darle la información de cuantos bytes tiene que incrementar o decrementar.

Entre las líneas 17 y 47 se define la función main, en esta se genera un lazo para recorrer un menú de opciones, y se selecciona la función correspondiente mediante una estructura switch-case. Para salir se utiliza la función exit con un parámetro 0 que indica una salida normal del programa.

Desde la línea 49 a 72 se define la función ingresa, su prototipo esta ubicado en la línea 12, recibe como parámetro la dirección del puntero al comienzo de la lista (un puntero a puntero a struct Lis llamado Principio) y devuelve void.

Se declaran dos punteros (Actual y Nuevo) a struct Lis que se utilizaran para guardar los valores de los punteros de los elementos de la lista, a Nuevo se lo utilizara para apuntar al nuevo elemento, a Actual para que apunte al elemento actual en la lista.

En la línea 53 se solicita memoria para ubicar al nuevo elemento mediante la función malloc, esta devuelve el valor del bloque al puntero Nuevo y se pregunta por el resultado del pedido en caso de que hubiese devuelto un NULL se informa el echo y luego se retorna a main (líneas. 55 a 59).

En el caso de que el resultado del requerimiento de memoria fuese satisfactorio se ingresara el nuevo dato (línea 62).

En la línea 63 se le asigna al puntero Nuevo->Sig el valor NULL ya que va a ser el último elemento de la lista.

En la línea 64 se pregunta por el valor del contenido del puntero a puntero Principio, en caso de que este sea NULL indica que la lista esta vacía y por lo tanto se le asigna a este el valor de Nuevo y termina la función retornando a main (líneas 66 y 67).

Si no fuese de así quiere decir que hay por lo menos un elemento en la lista y por lo tanto hay que proceder a buscar el lugar en donde se lo agregara, esto se realiza entre las líneas 69 y 70. Como primer paso, se asignara al puntero Actual el valor del contenido del puntero a puntero Principio (línea 51). Entre las líneas 69 y 70 se ejecuta un lazo del tipo while con la condición (Actual->Sig), que verifica que el puntero Actual->Sig no sea NULL; dentro del lazo se actualiza el valor del puntero Actual, a Actual se le asigna el valor de Actual->Sig, es decir el valor del elemento siguiente. De tal manera se recorrerá la lista hasta encontrar el fin de la lista para agregar el nuevo elemento.

Al llegar al final el valor de Actual->Sig será NULL y por lo tanto el nuevo elemento se va a ubicar allí. Esto se realiza asignándole al puntero Actual->Sig el valor del puntero Nuevo (línea 71), finalizando la ejecución de la función.

La función lee se define entre las líneas 74 y 85, esta recibe como parámetro la dirección del puntero a inicio de la lista en un puntero a puntero a struct Lis llamado Principio y devuelve void. El prototipo está en la línea 13.

Se define un puntero Actual al que se lo inicializa con el contenido de Principio (línea 76). Este puntero servirá para recorrer la lista. La lista se recorre mediante un lazo while con la condición de que el puntero Actual sea distinto de NULL (líneas 78 a 82). Dentro del lazo se imprimen los valores (línea 80), en la línea siguiente se modifica el valor de Actual a Actual->Sig para ir al elemento siguiente en la lista. Al finalizar este recorrido se retornara a main.

En las líneas 87 a 115 se define la función borra, su prototipo esta en la línea 14. En ella se declaran dos punteros Actual y Anterior, inicializándolos con la dirección de inicio de la lista, estos se van a utilizar para recorrer la lista.

En las líneas 92 y 93 se ingresa el valor que se quiere eliminar. Luego se comienza a recorrer la lista para encontrarlo mediante un lazo, los punteros Actual y Anterior están inicializados con el contenido de Principio, después de esto se ingresa a un ciclo while con la condición Actual&&(Valor!=Actual->Dato), se recorrerá la lista hasta encontrar el valor deseado o que el puntero Actual sea distinto de NULL, en el cuerpo del while se modifican los valores de los punteros Actual y Anterior, al primero se le asigna el valor Actual->Sig para poder recorrer la lista y al segundo el valor de Actual para no perder el lugar del elemento anterior (líneas 94 a 98).

Si se encuentra que el valor del puntero Actual no es NULL quiere decir que se ha encontrado el valor, si además si el contenido de Principio es igual al de Actual quiere decir que el elemento es el primero, se procede a modificar el contenido de Principio con el puntero Actual->Sig y luego se libera la memoria utilizando free(), retornando a main (líneas 99 a 104). Si ocurre que Actual es distinto al contenido de Principio significa que es un elemento distinto al primero actualizando el puntero Anterior->Sig con el valor de Actual->Sig y luego se libera la memoria con la función free, retornando a main (líneas 105 y 110).

Desde la línea 111 a 114 se indica que el dato deseado no se ha encontrado.

Desde la línea 117 a 126 se encuentra definida la función Elimina que se utiliza para eliminar los elementos que quedaron en la lista para devolver al sistema operativo los bloque de memoria solicitados antes de la finalización del programa. Su prototipo se encuentra en la línea 15.

Esta función recorre la lista modificando el puntero Principio y liberando la memoria mediante el uso del puntero Actual.

```
1. void Ingresa(struct Lis **Principio)
2. {
3.     struct Lis *Actual,*Nuevo,*Anterior;
4.     if(!(Nuevo=(struct Lis *)malloc(sizeof(struct Lis))))
5.     {
6.         printf("\n\nNo hay memoria disponible\n");
7.         printf("\n\nPresione una tecla para continuar\n");
8.         getch();
9.         return;
10.    }
11.    printf("\n\nIngresa el dato : ");
12.    scanf("%d",&Nuevo->Dato);
13.    if(!*Principio)
14.    {
15.        *Principio=Nuevo;
16.        Nuevo->Sig=NULL;
17.        return;
18.    }
19.    Anterior=*Principio;
20.    Actual=*Principio;
21.    while(Actual&&(Actual->Dato>Nuevo->Dato))
22.    {
23.        Anterior=Actual;
24.        Actual=Actual->Sig;
25.    }
26.    if(!Actual)
27.    {
28.        Anterior->Sig=Nuevo;
29.        Nuevo->Sig=NULL;
30.        return;
31.    }
32.    if(Anterior==Actual)
33.    {
34.        *Principio=Nuevo;
35.        Nuevo->Sig=Anterior;
36.        return;
37.    }
38.    Anterior->Sig=Nuevo;
39.    Nuevo->Sig=Actual;
40. }
```

La función ingresa escrita difiere de la función ingresa que esta en el programa anterior en que en esta ultima cada nuevo elemento se inserta en la lista en la ubicación correspondiente a fin de que la lista siempre quede ordenada.

Ello se hace entre las líneas 13 a 39, en la 13 se verifica si el contenido del puntero Principio es NULL, esto significa que la lista esta vacía. En la línea 15 se apunta el

inicio de la lista al elemento Nuevo, en la 16 se le asigna al puntero Nuevo->Sig el valor NULL para indicar que es el ultimo elemento y en la 17 se retorna a la función main.

En las líneas 19y 20 se inicializan los punteros Anterior y Actual con el inicio de la lista, para comenzar a recorrerla y encontrar la posición en donde se ubicara el nuevo elemento.

Entre las líneas 21 y 25 se comienza a recorrer la lista para poder encontrar la posición en donde se insertara el nuevo elemento. Esto se realiza mediante un lazo while con la condición Actual&&(Actual->Dato>Nuevo->Dato), esta condición hace que el lazo finalice cuando se encuentre el fin de la lista o se halla encontrado que el elemento al que apunta Actual contenga un dato mayor que el que se quiere insertar. En las líneas 23 y 24 se actualizan los valores de los punteros Anterior y Actual. A Anterior se le asigna el valor que tiene el puntero Actual de tal forma que este apunte al elemento anterior para que con este se pueda enlazar el nuevo elemento. Al puntero Actual se le asigna la dirección del elemento siguiente para poder encontrar el fin de lista o la ubicación del elemento que apuntara el nuevo elemento para poder enlazar a la lista.

Entre las líneas 26 a 31 se verifica que se llego al fin de la lista, esta situación se obtiene mediante la evaluación del puntero Actual, si este tiene el valor NULL, indica que se llego al fin de la lista. En esta situación se le asigna a Anterior->Sig la dirección del nuevo elemento (línea 28). Y como el nuevo elemento es el ultimo a su puntero Sig se le asigna el valor NULL para indicar el fin de la lista (línea 29) retornando a la función main.

Desde la línea 32 a 37 se verifica que el elemento se insertara en el primer lugar de la lista, esta condición se obtiene analizando el estado de los punteros Anterior y Actual, si estos son iguales indica que no se ha comenzado a recorrer la lista, por lo que el nuevo elemento se insertara en el primer lugar. Esto se realiza asignándole al contenido de Principio la dirección del nuevo elemento (línea 34). Para enlazar la lista a Nuevo->Sig se le asigna el valor del puntero Anterior que contiene la ubicación del elemento que era el inicio de la lista (línea 35) y se retorna a main.

Entre las líneas 38 y 39 se ubica al elemento en una posición intermedia en la lista asignándole al puntero Anterior->Sig la dirección de nuevo y a Nuevo->Sig la dirección del puntero Actual enlazando la lista.